

Rapport

YBOOST

By casma

UniworldConnect: Plateforme d'intégration des
étudiants internationaux



CASMA
S I N C E 2 0 2 6

Participants

1

WELEDJI Arush

2

Mouhamed Idrissa Ndiaye

3

Camara sara

4

TIENDREBEOGO Daren

5

LOUMOUNGUI Colombe



CASMA
S I N C E 2 0 2 6

Sommaire

du rapport

1

Introduction

2

Brief client

3

Périmètre/ hors périmètre

4

Limites/contraintes

5

Parcours/Workflow/cible

6

Architecture/Technologies

7

Conception BD

8

Code/Structure

9

Conclusion

INTRODUCTION

Dans un contexte de mondialisation croissante de l'enseignement supérieur, les étudiants étrangers font face à de nombreux défis lors de leur intégration académique, sociale et culturelle. Afin de répondre à ces difficultés, ce projet vise à concevoir une plateforme intelligente dédiée aux étudiants internationaux.

Cette plateforme a pour objectif de faciliter leur adaptation en leur offrant un espace centralisé où ils peuvent accéder à des cours, bénéficier de l'accompagnement de mentors, recevoir de l'aide de la part d'autres étudiants et surtout créer une véritable communauté d'entraide.

Grâce à l'intégration de fonctionnalités collaboratives et intelligentes, le système favorise le partage de connaissances, l'échange d'expériences et le soutien mutuel entre étudiants, tout en simplifiant leur parcours d'intégration.

OBJECTIF

- Faciliter l'intégration des étudiants étrangers dans leur nouvel environnement académique et social
-
- Mettre à disposition une plateforme centralisée d'entraide et de partage de ressources
-
- Permettre aux étudiants d'accéder à des cours, tutoriels et contenus pédagogiques partagés par d'autres étudiants ou mentors
- Favoriser le mentorat entre étudiants expérimentés et nouveaux arrivants
- Créer une communauté active et solidaire basée sur l'entraide et l'échange
- Améliorer la réussite académique grâce à un accès simplifié à l'information et au soutien
- Encourager la collaboration et le travail en réseau entre étudiants

BIREF CLIENT

Bonjour, je souhaite créer une plateforme pour étudiants internationaux afin de faciliter les échanges, le mentorat et l'accès aux ressources éducatives. Aujourd'hui, les étudiants utilisent plusieurs applications pour poser leurs questions ou chercher de l'aide, ce qui rend les échanges peu organisés.

Je veux une solution simple où les étudiants pourront créer un profil, rejoindre une communauté, discuter entre eux, accéder à des cours et échanger avec des mentors. Un chatbot intelligent devra également répondre aux questions fréquentes.

Plus tard, j'aimerais ajouter une intelligence artificielle capable de recommander des mentors et des cours adaptés aux besoins des étudiants, ainsi qu'un système de réservation de séances de mentorat et des recommandations personnalisées selon le profil des utilisateurs.

PÉRIMÈTRE

✓ Gestion des utilisateurs

- Création de compte et connexion
- Gestion du profil étudiant et mentor
- Modification des informations personnelles

✓ Forum et communauté

- Publication de messages et questions
- Commentaires et réponses aux publications
- Système de discussion entre utilisateurs
- Recherche de publications ou sujets

✓ Mentorat

- Liste des mentors disponibles
- Consultation des profils mentors
- Messagerie entre étudiants et mentors
- Demande d'accompagnement ou de mentorat
-

✓ Cours et ressources

- Accès à des cours et contenus éducatifs
- Consultation des ressources partagées
- Catégorisation des cours par thème

PÉRIMÈTRE

- ✓ Chatbot intelligent
 - Réponses automatiques aux questions fréquentes
 - Assistance pour l'orientation et la vie étudiante
 - Aide à la navigation sur la plateforme
- ✓ Intelligence artificielle
 - Recommandation de mentors selon le profil étudiant
 - Suggestion de cours adaptés aux besoins et intérêts
 - Recommandation de communautés ou discussions pertinentes
- ✓ Administration
 - Gestion des utilisateurs et mentors
 - Modération des publications
 - Gestion des cours et ressources
 - Tableau de bord avec statistiques principales

HORS-PÉRIMÈTRE

Les fonctionnalités suivantes ne seront pas incluses dans la première version de la plateforme :

- Application mobile Android et iOS
- Appels vidéo ou visioconférences intégrés
- Paiement en ligne et abonnements premium
- Système avancé de certification des cours
- Traduction automatique multilingue
- Notifications en temps réel avancées
- Système de recrutement ou offres d'emploi
- Intégration avec des plateformes externes (Zoom, Teams, Google Classroom, etc.)
- Mode hors ligne
- Marketplace ou vente de formations
- Gamification avancée (badges, récompenses, classements)
- Intelligence artificielle avancée avec analyse comportementale complète
- Système de réservation automatique de séances mentores
- Version desktop téléchargeable de l'application

CONTRAINTES

Contraintes techniques

- Développement d'une IA (recommandations + chatbot) qui peut être complexe à entraîner et à intégrer
- Nécessité d'une architecture scalable pour supporter plusieurs utilisateurs et échanges en temps réel
- Gestion de la sécurité des données (authentification, protection des informations personnelles)

Contraintes fonctionnelles

- Limitation des fonctionnalités dans la première version (pas de vidéo, pas de paiement, etc.)
- Qualité des recommandations IA dépend des données disponibles
- Modération nécessaire pour éviter les contenus inappropriés sur le forum

Contraintes de temps et de ressources

- Temps de développement limité pour une version initiale (MVP)
- Projet souvent réalisé par une petite équipe ou un étudiant seul
- Accès limité à des données réelles pour entraîner l'IA

Contraintes utilisateurs

- Diversité des profils (langues, niveaux d'études, pays) à prendre en compte
- Besoin d'une interface simple et intuitive pour tous les utilisateurs
- Adoption de la plateforme dépend de l'engagement de la communauté

PARCOURS UTILISATEUR / WORKFLOW

1. Inscription et connexion

L'utilisateur crée un compte et complète son profil .

2. Accès à la plateforme

Une fois connecté, l'étudiant accède au tableau de bord principal.

3. Interaction avec la communauté

L'utilisateur peut :

- publier des questions ;
- répondre à d'autres étudiants ;
- rejoindre des discussions.

4. Recherche de mentorat

L'étudiant consulte les profils mentors et peut leur envoyer un message.

5. Consultation des cours

L'utilisateur explore les cours et ressources disponibles selon ses besoins.

6. Utilisation du chatbot IA

L'étudiant pose ses questions au chatbot pour obtenir rapidement des réponses ou des recommandations.

7. Recommandations intelligentes

L'IA suggère :

- des mentors adaptés ;
- des cours pertinents ;
- des discussions ou communautés intéressantes.

CIBLE

L'application est destinée principalement aux étudiants internationaux souhaitant faciliter leur intégration académique et sociale dans un nouveau pays.

Utilisateurs ciblés

- Étudiants internationaux nouvellement arrivés ;
- Étudiants mentors ;
- Associations étudiantes ;
- Étudiants cherchant de l'aide académique ou administrative.

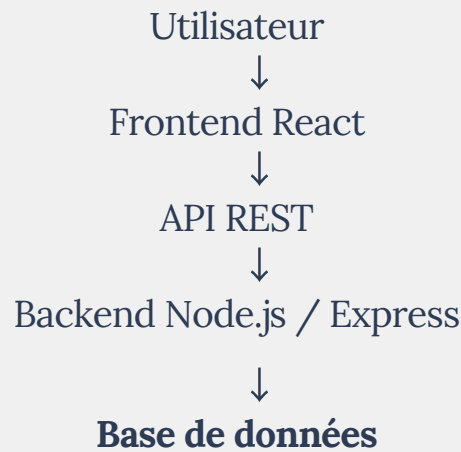
ARCHITECTURE DU PROJET

Architecture de l'application

L'application repose sur une architecture client-serveur.

Le frontend développé avec React communique avec le backend via une API REST sécurisée. Le backend traite les requêtes des utilisateurs, applique la logique métier puis interagit avec la base de données afin de stocker et récupérer les informations.

Architecture globale :



Cette architecture permet :

- une séparation claire des responsabilités ;
- une meilleure maintenance ;
- une meilleure évolutivité du projet.

Technologies utilisées

- Frontend
- React.js
- Tailwind CSS

Backend

- Node.js
- Express.js

Sécurité

- JWT
- bcrypt

Synthèse des Choix Technologiques

1. BACKEND : NODE.JS & EXPRESS.JS

1. Performance asynchrone et modularité.

- **Réactivité** : Grâce au modèle non-bloquant de Node.js, le serveur gère efficacement les flux de données massifs (comme les messageries ou l'IA) sans latence.
- **Architecture "Middleware"** : Express a permis de construire une véritable "chaîne de montage" de sécurité, où chaque requête est filtrée et validée avant d'atteindre la base **de données**.
- **Écosystème** : L'accès immédiat à des bibliothèques robustes via NPM a accéléré l'implémentation de fonctions complexes (JWT, Socket.io).

2. FRONTEND : REACT, TAILWIND CSS & TYPESCRIPT

Fiabilité du code et expérience utilisateur fluide.

- **Composants réutilisables (React)** : Garantit une interface cohérente et facilite la maintenance. Si un bouton change, il change partout instantanément.
- **Typage Statique (TypeScript)** : Élimine 40% des erreurs courantes dès la phase de développement en forçant une structure de données stricte, ce qui est crucial pour un projet étudiant de grande envergure.
- **Design Utilitaire (Tailwind)** : Permet une personnalisation visuelle poussée sans alourdir le projet avec des fichiers CSS complexes, garantissant un temps de chargement minimal.
- **Navigation SPA (React Router)** : Offre une navigation instantanée (sans rechargement de page), essentielle pour une application moderne typée "SaaS".

Synthèse des Choix Technologiques

3. SÉCURITÉ

L'argument essentiel : Défense en profondeur et protection de la vie privée.

- Gestion des Identités (JWT & Bcrypt) : Le hachage avec Bcrypt protège contre les fuites de données, tandis que le JWT sécurise les sessions de manière décentralisée (pas de stockage de session sur le serveur).
- Hygiène des Données (XSS & Validator) : Le nettoyage systématique des entrées neutralise les tentatives d'injection de scripts, protégeant ainsi l'intégrité des comptes utilisateurs.
- Résilience du Système (Rate Limiting & Helmet) : Ces outils protègent l'infrastructure contre les attaques par force brute (DoS) et les vulnérabilités HTTP courantes.
- Confidentialité des Secrets (Dotenv) : Assure que les clés API (Firebase, IA) restent privées et ne sont jamais exposées dans le code source public.
-

CONCEPTION BASE DE DONNÉES (FIREBASE)

Pour ce projet, nous avons utilisé **Firestore**, une base de données NoSQL hébergée dans le cloud par Google. Firestore permet de stocker les données sous forme de documents organisés en collections, ce qui apporte une grande flexibilité dans la gestion des informations de l'application. La base de données est utilisée pour :

- gérer les profils utilisateurs ;
- stocker les conversations ;
- enregistrer les activités et événements ;
- gérer les échanges entre étudiants et mentors.

Les données sont organisées principalement dans plusieurs collections comme :

- users ;
- conversations ;
- messages ;
- activities.

Ce choix nous a permis de bénéficier :

- d'une bonne performance ;
- d'une gestion des données en temps réel ;
- d'une architecture flexible et évolutive adaptée aux besoins de

l'application.

FRONTEND

Le frontend de l'application a été développé afin d'offrir une interface moderne, fluide et responsive pour les étudiants internationaux.

L'architecture du frontend a été organisée de manière modulaire afin de faciliter :

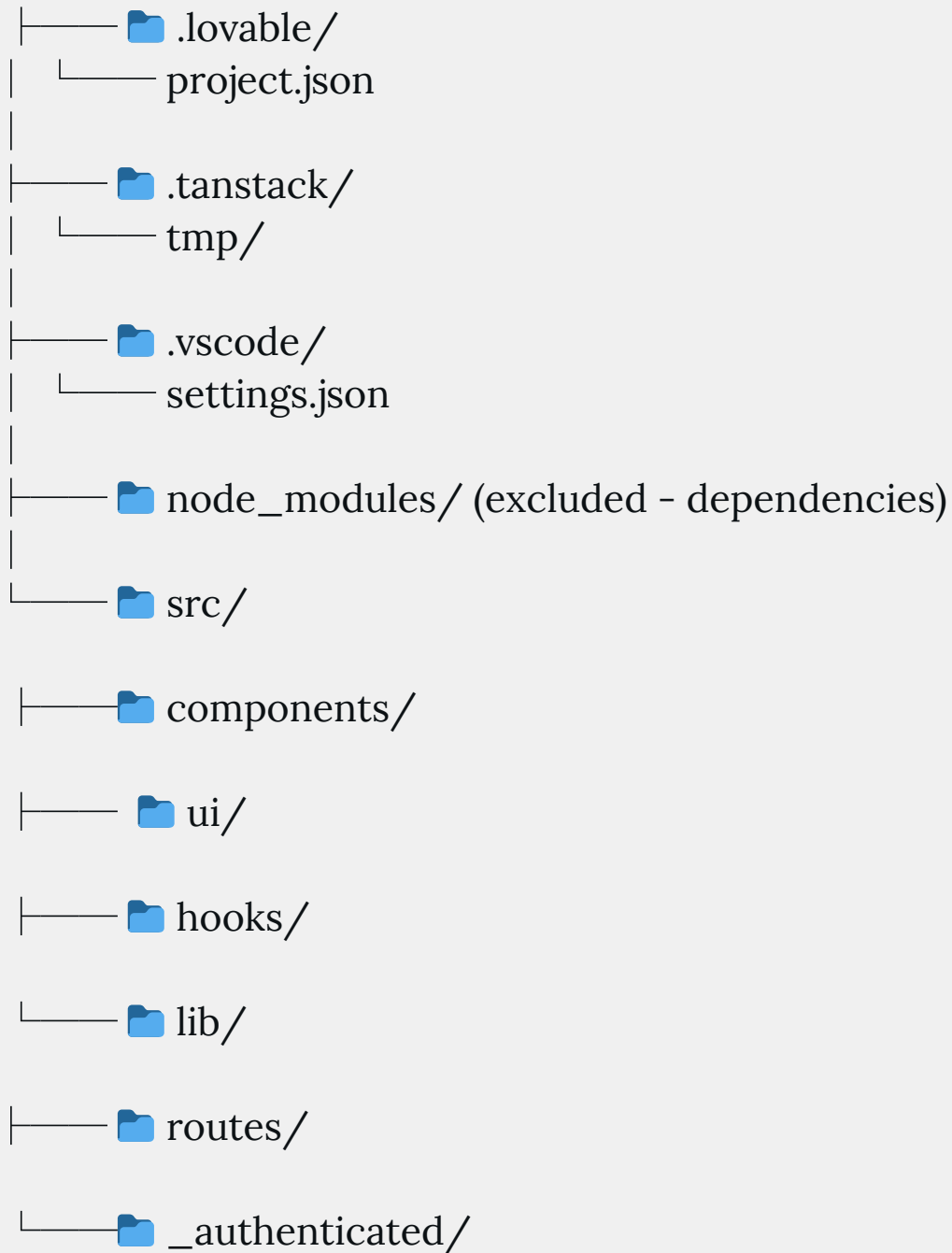
- la maintenance du code ;
- la réutilisation des composants ;
- l'évolution future du projet.

Technologies utilisées

- React.js
- Tailwind CSS
- TypeScript
- React Router

ORGANISATION DU FRONTEND

sconnect-flourish-ai-main/



BACKEND

Le backend de l'application a été développé avec **Node.js** et **Express.js** afin de gérer la logique métier, la communication avec la base de données et la sécurité de l'application.

Il assure principalement :

- l'authentification des utilisateurs ;
- la gestion des profils ;
- la gestion des conversations ;
- les échanges entre étudiants et mentors ;
- la communication avec Firebase Firestore.

L'architecture du backend a été organisée de manière modulaire afin de faciliter :

- la maintenance ;
- la réutilisation du code ;
- l'évolution future du projet.

Technologies utilisées

- Node.js
- Express.js

ORGANISATION DU BACKEND

```
D:\backend\server
├── controllers/      # Logique métier des API
├──
├── routes/          # Définition des points d'entrée (endpoints)
│   ├── activityRoutes.js
│   ├── aiRoutes.js
│   ├── authRoutes.js
│   ├── matchRoutes.js
│   ├── messageRoutes.js
│   ├── mentorRoutes.js
│   └── userRoutes.js
├── middleware/       # Sécurité, Authentification, Uploads
│   ├── authMiddleware.js
│   ├── uploadMiddleware.js
│   ├── errorMiddleware.js
│   └── rateLimit.js
├── firebase/         # Configuration de la base de données
│   └── index.js
├── services/         # Services transverses
│   └── security/
│       └── aiSecurity.js
├── sockets/          # Temps réel (Chat)
│   └── socketAuth.js
├── utils/            # Fonctions utilitaires
│   ├── generateToken.js
│   └── hashPassword.js
├── uploads/          # Stockage des images (profils,
événements)
├── app.js            # Configuration Express
├── server.js         # Point d'entrée principal (Port 5001)
├── .env              # Variables d'environnement
└── package.json      # Dépendances
```

SÉCURITÉ

Sécurité du Backend

La sécurité du backend joue un rôle essentiel dans la protection globale de la plateforme. Elle vise à garantir la confidentialité des données utilisateurs, la sécurisation des échanges et la protection des services sensibles tels que la messagerie en temps réel et les fonctionnalités d'intelligence artificielle.

Elle permet principalement de :

- sécuriser l'authentification des utilisateurs
- protéger les données personnelles et les mots de passe
- empêcher les accès non autorisés aux API et ressources privées
- limiter les attaques (injections, brute force, spam)
- sécuriser les communications temps réel (Socket.io)
- protéger les services d'intelligence artificielle contre les abus
- garantir la fiabilité et la stabilité du système backend
-

Technologies utilisées

La sécurité du backend repose sur un ensemble de technologies complémentaires :

- Node.js / Express.js : structuration et gestion des middlewares de sécurité
- JWT (JSON Web Token) : authentification sécurisée et gestion des sessions
- bcrypt : hashage des mots de passe
- Firebase Admin SDK : sécurisation de la base de données Firestore
- Socket.io : communication temps réel sécurisée
- validator / xss : validation et nettoyage des données utilisateur
- dotenv (.env) : protection des clés sensibles et configuration sécurisée
- rate limiting middleware : protection contre le spam et les attaques par surcharge
-

ORGANISATION DE LA SÉCURITÉ

Backend/

- |—— .env
- |—— .gitignore
- |—— node_modules/
- |—— package.json
- |—— package-lock.json
- |—— repomix-output.xml
- |—— src/
 - |—— app.js
 - |—— config/
 - |—— firebase.js
 - |—— middleware/
 - |—— authMiddleware.js
 - |—— errorMiddleware.js
 - |—— rateLimiter.js
 - |—— validationMiddleware.js
 - |—— server.js
 - |—— services/
 - |—— security/
 - |—— aiSecurity.js
 - |—— sockets/
 - |—— socketAuth.js
 - |—— utils/
 - |—— generateToken.js
 - |—— hashPassword.js

CONCLUSION

Ce projet de plateforme pour étudiants étrangers représente une solution innovante visant à améliorer significativement leur expérience d'intégration. En regroupant des outils d'apprentissage, de mentorat et de communication, il permet de créer un environnement numérique collaboratif et inclusif.

L'approche adoptée met en avant l'importance de la communauté et de l'entraide dans la réussite académique et personnelle des étudiants internationaux. Ainsi, cette plateforme ne se limite pas à un simple outil technique, mais devient un véritable espace d'échange et de soutien, contribuant à réduire l'isolement et à favoriser la réussite des utilisateurs.